BigData - Baseball Hall of Fame ballot prediction

Rasmus Faber

Januar 2026

1 Introduction

1.1 Problem Statement

The National Baseball Hall of Fame of the United States of America (USA) represents the top of professional achievement in Major League Baseball (MLB). For a lot of players, it is a dream, to reach this level of fame. However, it is a dream which is not easy to achieve. The journey there is strictly regulated by the Baseball Writers' Association of America (BBWAA).

Before getting in there, a player has to first of get his name on a ballot, which makes him eligible to be voted into the HoF. To get on the ballot, he must pass through filter of eligibility [1].

1. The player must have played in parts of at least 10 Major League seasons
2. He is only eligible for the ballot five years after his final game
3. A Screening Committee of six BBWAA members look for players who had performances and a reputation of a star player

Given comprehensive per-season performance statistics from the Lahman Baseball Database [2] spanning 1871-2023 (Batting.csv, Pitching.csv, Fielding.csv, Salaries.csv, all-star.csv), the problem to solve which was chosen is to develop a binary classification model that takes over the role of the Screening Committee and predicts whether a given player deserves to appear on a Hall of Fame ballot. The target variable is "ballot_appearance", where a 1 implies that the player does appear and a 0 that he does not.

This problem presents several challenges: (1) extreme class imbalance due to ballot rarity, (2) multi-table joins across thousands of player-season records, (3) heterogeneous features and (4) temporal dependencies requiring career-level aggregation.

1.2 Project Objective

The objective is to implement a complete PySpark ML pipeline.

- Perform comprehensive EDA with 15–20 KPIs

- Engineer career-aggregate features (total AB, H, IPouts, salary-adjusted WAR, position tenure)
- Test 5 classifiers, best 3 with grid search hyperparameter tuning
- Evaluates using AUROC and macro F1-score, as well as confusion matrices on 80/20 train-test splits

The pipeline demonstrates big data scalability, handles real-world data quality issues, and provides interpretable insights into Hall of Fame ballot selection criteria.

2 Data Exploration & Understanding

2.1 Data Sources

As the data source for this project, the Lahman Baseball Database from Kaggle was chosen [2]. The data from this database spans the years 1871-2023 and contains data on most events, teams and players during this time. The following tables were selected from the database:

- Batting table
- Pitching table
- Fielding table
- Salaries table
- All-Star table

These tables were deemed as relevant, as only players are being considered in this version of the model (no trainers or other staff).

2.2 Feature Selection

Features were selected based on their relevance to Hall of Fame ballot criteria, following sabermetric principles of objective baseball analysis [3]. Starting with batting stats like runs-batted-in (RBI), hits (H) or homeruns (HR). Additionally, also the fielding stats put-outs (PO), assists (A) and errors (e) were selected as features, in order to consider the fielding performance of players.

Also, after consideration, pitchers were also selected, so pitching stats such as earned-run-average (ERA), wins (w) and losses (l). Of course, wenn choosing the pitcher to participate, it was also important to include the role of the player, as it plays a big role in how stats need to be assessed.

Finally, two more features were selected in the form of the salary, and the amount of all-star-games played. All features chosen can be assessed here:

In total the joined dataframe with all the data from the different tables contained 214.157 rows and 75 columns.

Category	Abbreviation	Statistic Definition
Batting	G	Games Played
	AB	At Bats
	H	Hits
	HR	Home Runs
	RBI	Runs Batted In
	BB	Base on Balls (Walks)
	SO	Strikeouts (Batting)
Pitching	W	Wins
	L	Losses
	ERA	Earned Run Average
	IPOuts	Outs Pitched (Innings Pitched $\times$ 3)
	Pit_SO	Strikeouts (Pitching)
Fielding	PO	Putouts
	A	Assists
	E	Errors
General	Salary	Total Career Earnings
	POS	Primary Position
	All_Star	Total All-Star Appearances

Table 1: Player Career Statistics and Definitions

2.3 Data Preprocessing & Engineering

In order to preprocess the data and prepare it for further analysis, multiple steps were necessary.

First, a manual step was performed to select the columns deemed relevant for the problem (these are described in the previous section). With these features, 20 columns remained in the dataframe.

The next step was to create the column "ballot_appearance", which indicates whether a player was ever on the ballot for the HoF during or after their career. This new column was then joined to the dataframe of features.

Another important feature that needed to be engineered was the "position". When transforming the data from player-per-year to player-per-career level, there was the problem that players may play different positions during their career or even during the same year. This distorts some statistics and raises the question of which position to select for such a player. The approach taken here was to consider the position with the most put-outs in a single year as the main position of the player for that year. Over the entire career, the position that was the main position in most years was then considered the main position. This approach might of course have a bias effect on the model; however, as the games played were always the same, no more reliable statistic was available for this purpose.

In the next step, all the data was aggregated at the player-year level using only their main position. Afterwards, the data was aggregated at the player-

career level to obtain the total statistics of a player career.

```python
# Aggregation by player career stats for all years
player_stats = (
    labeledDF
    .groupBy("player_id", "ballot_appearance")  # one row per player
    .agg(
        # Batting totals over career
        spark_sum("g").alias("g"),
        spark_sum("ab").alias("ab"),
        spark_sum("h").alias("h"),
        spark_sum("hr").alias("hr"),
        spark_sum("rbi").alias("rbi"),
        spark_sum("bb").alias("bb"),
        spark_sum("so").alias("so"),

        # Pitching totals over career
        spark_sum("w").alias("w"),
        spark_sum("l").alias("l"),
        avg("era").alias("era"),
        spark_sum("ipouts").alias("ipouts"),
        spark_sum("pit_so").alias("pit_so"),

        # Fielding totals over career
        spark_sum("po").alias("po"),
        spark_sum("a").alias("a"),
        spark_sum("e").alias("e"),

        # Total career earnings is usually the strongest predictor
        spark_sum("salary").alias("salary"),

        # Primary position: first position observed
        first("pos").alias("pos"),

        # Total All-Star Games played
        spark_sum("all_star_games").alias("all_star_games")
    )
    .cache()
)

player_stats.show(5, truncate=False)
print(f"Player-level stats: {player_stats.count():,} rows")
```

Listing 1: Aggregation of Career Statistics by Player

Then, with the final columns all aggregated at the level that is intended, the dataframe was cleaned. When checking for duplicates, none were found, which makes sense due to the player_id as a unique identifier.

A bigger problem was the replacement of missing values. Columns, which

solely appear for pitchers (wins for example) had a lot of empty cells, for all the players who are not pitchers. Another column with a lot of empty cells was the salary column, which is a statistic only kept from 1985 onwards.

Table 2: Count of NULL Values per Column in Dataset

Category	Feature Column	NULL Count
Identifiers	player_id	0
	ballot_appearance	0
Batting	g (Games)	0
	ab (At Bats)	439
	h (Hits)	439
	hr (Home Runs)	439
	rbi (Runs Batted In)	574
	bb (Walks)	439
	so (Strikeouts)	1,975
Pitching	w (Wins)	9,533
	l (Losses)	9,533
	era (Earned Run Average)	9,561
	ipouts (Outs Pitched)	9,533
	pit_so (Pitching Strikeouts)	9,533
Fielding	po (Putouts)	201
	a (Assists)	202
	e (Errors)	203
Other	salary	13,831
	pos (Position)	351
	all_star_games	0

The first cells filled were the pitcher-specific ones. If the position of a player was not pitcher, all the values were filled with 0 as values.

```
# Replace missing values: Ensure pitching stats only apply
    to pitchers
# '1' or 'P' indicates the Pitcher position
player_stats = player_stats.withColumn("w",
    when(col("pos").isin("1", "P"), col("w")).otherwise(lit
    (0))) \
    .withColumn("l",
    when(col("pos").isin("1", "P"), col("l")).otherwise(lit
    (0))) \
    .withColumn("era",
    when((col("pos").isin("1", "P")) & (col("ipouts") > 0),
    col("era")).otherwise(lit(0)))
```

Listing 2: Handling Missing Values for Positional Statistics

Then, the remaining empty cells were filled. For salary, where a lot of cells were missing, a median salary was created for imputation, to dilute the role

that this stat had in the assessment of players without a salary written down. Another option to fill the empty cells was that when a player did not play any game, their fielding statistics were all going to be 0. If a player was at this point still without a position, his position was marked as unknown. After that, all the other cells remaining empty were filled with zeros. The preprocessed and cleaned dataframe could then be used for the data analysis.

For the model later on, the data was still transformed. The position was one-hot-encoded, and the ballot_appearance was turned into the label. A vector assembler was used to put all features in a vector together.

Then, the data was standard scaled. This was supposed to help to avoid large values dominating. As Salary could have very large values, it was decided to take this step, to give more importance to the other values.

Another important thing which was done in this section was to use class weighting in order to try and solve the problem of one class being severely underrepresented. To achieve this, a balancing ratio was applied to the positive class.

No outliers were removed in the data preprocessing process. The explanation for this was that due to the exceptional achievements which were necessary to get on the ballot, removing outliers would lead to a lot of positive results being removed, which would even further reduce the role of the already underrepresented class.

2.4 Descriptive Statistics

The data in the dataset is very unbalanced. There are only very few players, who get selected into the HoF. There are more, who get chosen for the ballot, but in total just about 6% of all players ever make it onto said ballot.

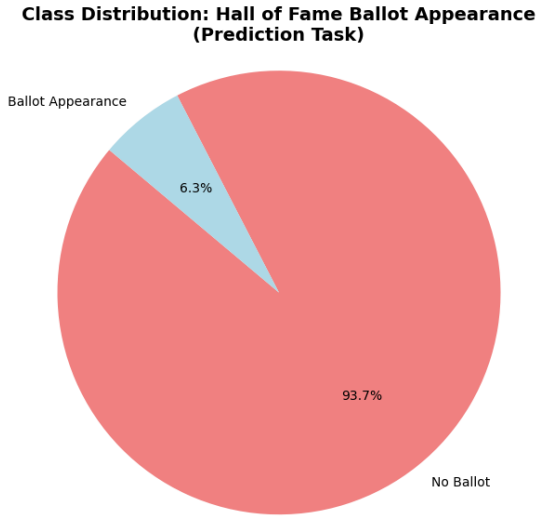

Figure 1: Class Distributions

Another thing, which can be said, is that players are distributed very unequally between the different positions. By far most players are Pitchers (P), the second most frequent role is Outfielder (OF). The positions being played the least are the the specific outfield positions (LF, RF, CF) and the designated hitter (DH).

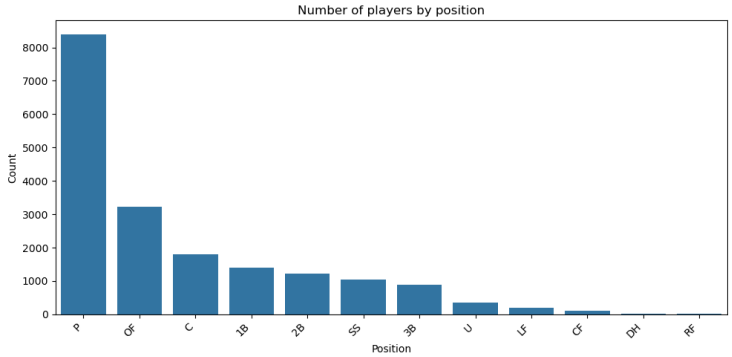

Figure 2: Distribution over positions.

The fact, that there are so many pitchers, does not increase the chances of a pitcher being on the HoF ballot however. They are the position with the second-lowest probability of getting on the ballot. Lower is only the position of unknown (U), which shows that a player was not considered important, if there is not even a role documented for him.

On the other hand, right fielders have almost a 40% chance of making it onto the ballot. This is most likely due to the fact, that there are not a lot such players. Also, outfield positions are mostly considered to not be so important for fielding, and more for hitting. As in general hitting statistics are considered to be more important in the HoF ballot review, this might be a reason for a higher percentage for outfielders in general and also designated hitters.

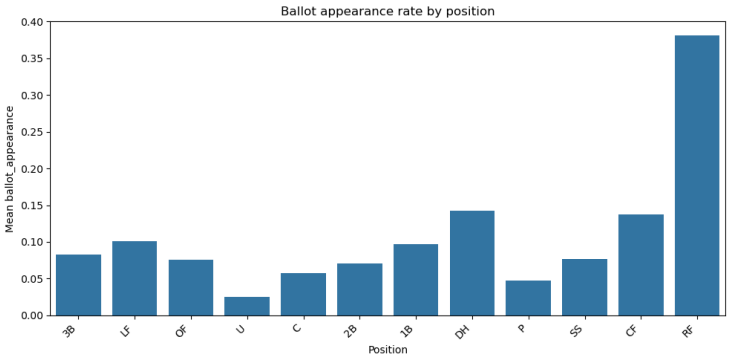

Figure 3: Chance of player from position appearing on ballot.

In general, this tendency and bias towards hitting statistics over fielding statistics can also be observed in the following graph, as it shows clearly that directly after the total games played, the most important statistics are hits and runs-batted-in, while put-outs and assists have a lot less correlation with appearing on the ballot.

Another thing which catches the eye in the graphic is the fact that salary apparently has the lowest correlation out of all the attributes (disregarding ERA, which is important for pitchers). However, after trying to leave it out, it showed that it does contain important insights especially for players from the last years, as the salary has only recently been measured.

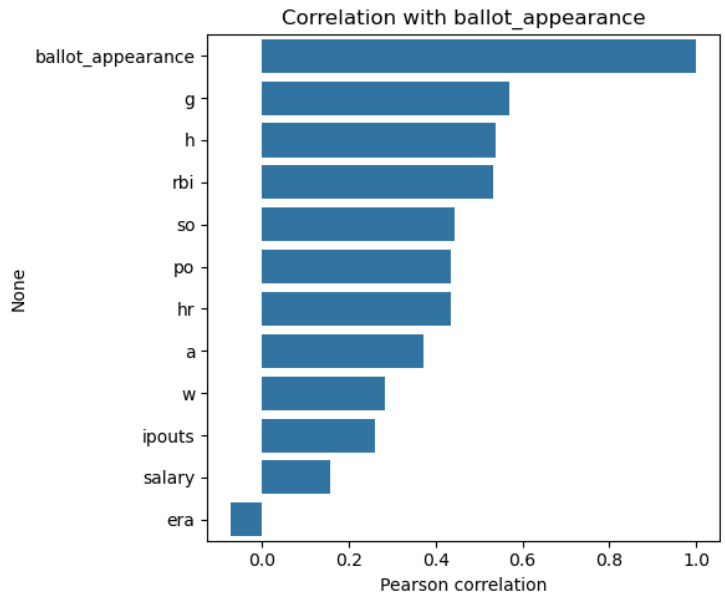

Figure 4: Correlation of different stats with ballot appearance

3 Modeling & Methodology

3.1 Model Selection

For the task, classification models were selected. As specified in the project description, five different models were chosen. The chosen classifiers were the following:

- Logistic Regression
- Decision Tree
- Random Forest
- Gradient Boosting
- Neural Network

3.2 Training Strategy

First off, the five selected models were tested with standard values. Their performance was assesed in regards to multiple metrics. As the decisive metric, for which model to use for the fine-tuning process later, the AUROC was selected, as it does present well which model performs good, keeping in mind the class imbalance.

There, the three models chosen for the hyperparameter tuning are the MLP, the gradient boosting classifier, and the best performing model: the random forest classifier, with an AUROC of 0.9728.

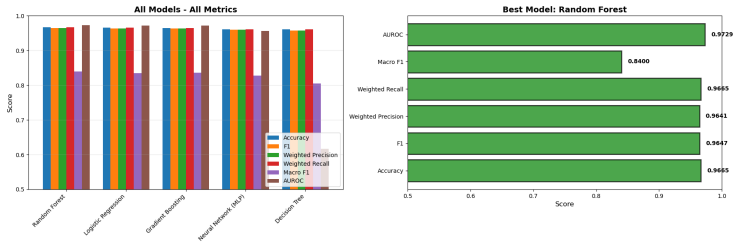

Figure 5: Performance of models

3.3 Hyperparameter Tuning

For the hyperparameter tuning, a grid-search with cross-validation is performed. The goal is to optimize the top three performing models identified in the initial evaluation: the Random Forest classifier, Gradient Boosting classifier, and the Multilayer Perceptron (MLP).

For the Random Forest classifier, two key hyperparameters were tuned: the number of trees (numTrees) and the maximum depth (maxDepth) of each tree. The number of trees controls the ensemble size, so more trees generally reduce variance but increase the computational cost. The maximum depth constrains tree growth and helps to prevent overfitting. The grid search explored combinations of 50, 100, and 200 trees with maximum depths of 5, 10, and 15 levels, resulting in 9 parameter combinations.

This parameter range was chosen to balance model complexity with computational efficiency. Shallower trees (maxDepth=5) reduce overfitting risk, while deeper trees (maxDepth=15) capture more feature interactions. The grid search evaluates all combinations using stratified cross-validation with 3 folds, selecting the combination that maximizes the AUROC, which is appropriate for imbalanced classification tasks.

```
# Gridsearch with number of Trees and maxDepth
grid = ParamGridBuilder() \
    .addGrid(rfc.numTrees, [50, 100, 200]) \
    .addGrid(rfc.maxDepth, [5, 10, 15]) \
    .build()
```

Listing 3: Random Forest Hyperparameter Grid Setup

For the neural network (MLP classifier), three hyperparameters were tuned: the network architecture (layers), the maximum number of iterations (maxIter), and the learning rate (stepSize). The network architecture determines the representational power of the model. Three distinct architectures were evaluated:

a deeper, narrower network with four hidden layers; a standard medium architecture with two hidden layers; and a wider, deeper architecture combining both aspects. This variation allows exploration of different trade-offs between model complexity and generalization.

The maximum iterations parameter controls training duration. 100 iterations represents a lower baseline while 250 allows for more training. The step size (learning rate) controls gradient descent magnitude: a larger step size (0.03) permits faster convergence but risks instability, while a smaller step size (0.01) ensures more stable learning at the cost of slower convergence. Together, these parameters create 18 unique configurations.

```
# Setup the Parameter Grid for MLP
paramGrid = (ParamGridBuilder()
    .addGrid(mlp.layers, [
        [input_size, 24, 12, 8, output_size],   # Deeper, narrower architecture
        [input_size, 32, 16, output_size],       # Standard medium architecture
        [input_size, 48, 24, 12, output_size]    # Wider and deeper architecture
    ])
    .addGrid(mlp.maxIter, [100, 250])
    .addGrid(mlp.stepSize, [0.03, 0.01])
    .build())
```

Listing 4: Multilayer Perceptron (MLP) Parameter Grid Search

Finally, for the Gradient Boosting classifier, three hyperparameters were tuned: the maximum number of iterations (maxIter), the maximum tree depth (maxDepth), and the learning rate (stepSize). Unlike Random Forest which trains trees in parallel, Gradient Boosting builds trees sequentially, with each tree correcting errors from previous trees. The maximum iterations parameter determines how many boosting rounds to perform, ranging from 20 to 100 to balance model accuracy with computational cost.

The maximum depth controls individual tree complexity within the ensemble. Shallow trees serve as weak learners that reduce overfitting risk, while deeper trees capture more feature interactions but risk overfitting. The learning rate (step size) scales the contribution of each boosted tree: a higher rate accelerates learning but may cause instability, while a lower rate ensures more conservative updates. This grid search evaluates 18 configurations via cross-validationl.

```
# Setup the Parameter Grid for GBT
paramGrid = (ParamGridBuilder()
    .addGrid(gbt.maxIter, [20, 50, 100])
    .addGrid(gbt.maxDepth, [3, 5, 7])
    .addGrid(gbt.stepSize, [0.1, 0.05])
    .build())
```

Listing 5: Gradient Boosted Trees (GBT) Parameter Grid Search

4 Results & Evaluation

At the end of this process, the random forest classifier is once again the model with the highest AUROC. A feature importance analysis shows, that the most important features for the model, in order to decide whether a player deserves being on the ballot for the HoF, are the amount of games played, the amount of hits made, and the amount of all-star games they played. In general, it is once again very obvious, that hitting stats are the most important features for the models, besides the all-star games and overall games.

Table 3: Top 10 Most Important Features (Random Forest)

Rank	**Feature**	**Importance Score**
1	Games Played (g)	0.1442
2	Hits (h)	0.1102
3	All-Star Appearances	0.0910
4	At Bats (ab)	0.0884
5	Runs Batted In (rbi)	0.0784
6	Walks (bb)	0.0610
7	Putouts (po)	0.0527
8	Wins (w)	0.0465
9	Strikeouts (so)	0.0434
10	Errors (e)	0.0415

However, there was not really an optimization achieved by doing the grid-search, as the AUROC value remained stagnant. The macro-F1 score, which is highly important when evaluating classification performance for a problem with imbalanced data, rose up by 0.5%.

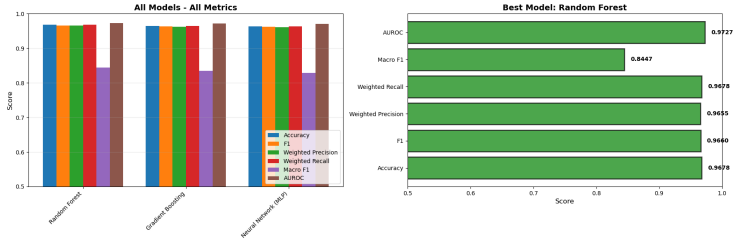

Figure 6: Performance of models after hyperparameter tuning

After achieving this result, the question was how to make the models relevant in the real-world, and actually useful. The important part here is, that the way that these models are currently, with their AUROC optimization, does not help a lot in their actual task. This is because, as can be seen for the random forest classifier here, the class which is actually important (the ones who appeared on a ballot) still has a very low rate of being predicted, of around 60%.

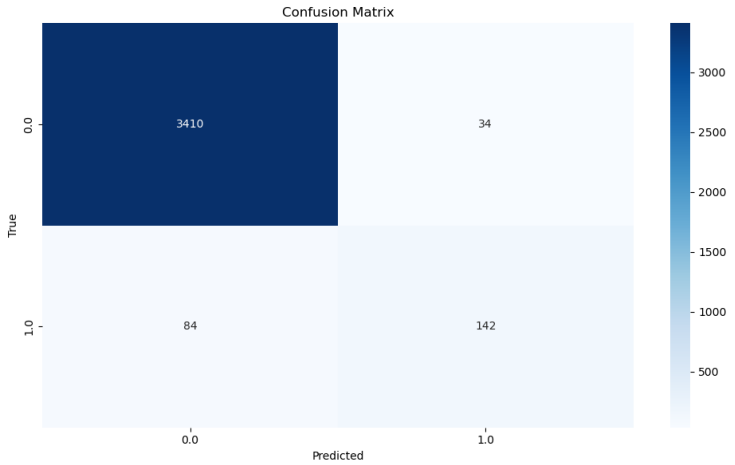

Figure 7: Random Forest classifier confusion matrix

So, to make the models practically useful, that rate has to be improved by a lot. It is a lot better, in order for the model to be useful in a pre-elimination step, that more people get flagged as ballot appearance worthy. The cost of having someone, who would deserve it not on the ballot is higher, than having someone who doesn't deserve it on the ballot. This is due to the real life consequences, as a bad player would never get voted into the HoF.

One approach to make the models more relevant was taken in this work. The threshold for separating the two classes was lowered by a lot, to have more players classified as ballot appearance worthy. The threshold chosen here was 0.1, which led to a lot more players being classified as appearing on the ballot.

The best-performing model in this case, considering the amount of correctly as ballot appearing classified players, is once again the random forest classifier, with 209 out of 226 ballot players successfully detected.

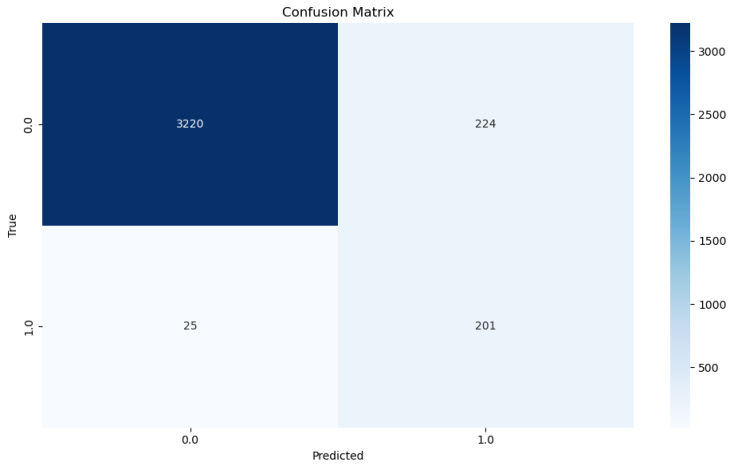

Figure 8: Random Forest threshold 0.1

5 Conclusion

Summarizing, it can be stated that machine learning models can provide a significant help for deciding the players, which should be put on the ballot for the baseball HoF. The models achieved a macro-F1 score of over 0.84, which shows that it does successfully get insights from the data.

However, as mentioned before, it can so far in this format only be used as a pre-decider. With a low threshold, the model will correctly identify the grand majority of ballot-appearance worthy players, while at the same time reducing the overall number of players who have to be assessed considerably. The final selection should still be done via the committee though, as there are also aspects involved in this process, which can simply not be measured by the data available here. One of the examples for this dilemma is the case of Pete Rose. He was a 17-times All-Star, which alone would probably get him an appearance on the ballot, but due to him betting on baseball games, including ones of his own team, he was permanently banned from appearing anywhere.

Without further insides on banned players, the data is limited in this case, as it will most likely never be able to understand, that the player with the most hits of all time does not even get an appearance on the ballot for the HoF.

References

[1] Baseball Hall of Fame, "BBWAA Election Rules," 2025. [Online]. Available: https://baseballhall.org/hall-of-fame/election-rules/bbwaa-rules

[2] S. Lahman, "Lahman Baseball Database," Kaggle Datasets, 2025. [Online]. Available: https://www.kaggle.com/datasets/dalyas/lahman-baseball-database.

[3] Society for American Baseball Research (SABR), "A Guide to Sabermetric Research," [Online]. Available: https://sabr.org/sabermetrics.